A3-PS022-TC177

eBPF/XDP Rate Limiting for DNS Query Floods in a Lab

Team TaskForce141

Naimish Pandey	naimishpandey82@gmail.com
Naitik Pandey	naitikpandey18@gmail.com
Dr. Atul Srivastava	asrivastava5@lko.amity.edu

AIORI Hackathon 3

October 10, 2026

Problem Description

Volumetric UDP DNS floods exhaust a Linux server's CPU before user space ever sees the packets.

- Every packet pays for sk_buff allocation and a walk through the netfilter/IP/UDP stack; at millions of packets per second the cores saturate in softirq context.
- Legitimate queries share the same RX queues and socket buffers, so they are delayed or dropped alongside the flood.
- nftables rate limits act after skb allocation: they cap what reaches the application but not the per-packet kernel cost.
- Goal: an XDP/eBPF filter on a single Linux host pair that does bounded UDP/DNS parsing and enforces per-source and global rate limits, benchmarked against no-filter and nftables baselines on three synthetic flood patterns.
- Metrics: throughput, CPU utilisation, packet drops and legitimate-query success rate.
- Scope: isolated lab only; no public flood testing, no full DNS semantic parsing, no universal PRSD detection, and no claim of eliminating reflection attacks.

Background

The concepts needed to follow the problem and the proposed optimisation.

- DNS over UDP/53 (RFC 1035): queries are tiny, stateless and fixed-format at the front (12-byte header), so they are cheap to generate in bulk and cheap to spoof.
- XDP runs eBPF bytecode at the earliest driver hook, before sk_buff allocation. Verdicts: XDP_PASS, XDP_DROP, XDP_TX, XDP_REDIRECT. Modes: native (driver), generic (skb-level, slower) and offload [1].
- The eBPF verifier requires bounded loops and explicit bounds checks on every packet access, which naturally enforces 'bounded parsing'.
- BPF maps provide state across packets: LRU hash maps for per-source buckets, per-CPU arrays for contention-free global counters and statistics.
- Token-bucket limiting: tokens refill at rate r up to burst b; a packet is admitted only if a token is available. This tolerates short legitimate bursts while capping sustained rates.

Literature & RFC Review

Only the most relevant studies and standards are listed.

Reference	What it contributes	Relevance to this project
XDP paper [1]	Describes the eXpress Data Path design and reports packet-drop performance far above the regular Linux stack.	Justifies dropping at the driver hook as the core idea.
Cloudflare XDP [2]	Production use of XDP/eBPF for L4 DDoS mitigation.	Shows the approach is practical; we focus on DNS rate limiting in a reproducible lab.
RFC 1035 [3]	Defines the DNS message and header layout.	Basis for the minimal, bounded header checks.
RFC 5358 [4]	Operational guidance on recursive resolvers abused as reflectors.	Frames scope: we limit load, we do not claim to stop reflection.
Kernel BPF docs [5]	Maps, helpers, verifier rules and XDP modes.	Implementation reference.

Gap: few lab-reproducible comparisons of per-source plus global XDP limits against nftables that report legitimate-query success, not just drop rates. This project fills that gap.

Proposed Solution

An XDP program on the server NIC, a load-generation harness on the client, and an automated benchmark matrix.

- Server host: XDP filter on the ingress NIC (native mode if supported, otherwise documented generic mode) in front of a stock DNS server.
- Client host: legitimate prober at a fixed low rate plus a flood generator.
- Three synthetic patterns: A single-source flood, B multi-source flood (many lab source addresses), C pulsed on/off flood with random synthetic labels.
- Configurations: no-filter, nftables (per-source hashlimit plus global limit), XDP.
- Loader/controller sets rates and reads per-CPU counters; metrics from mpstat, ethtool -S and bpftool.

Pseudocode

```
xdp_dns_ratelimit(ctx):
  parse eth (+1 VLAN tag), else PASS
  if not IPv4/UDP or fragment: PASS
  if udp.dport != 53: PASS
  if payload < 12 or > MAX_LEN: DROP
  if dns.QR == 1: PASS   # not a query
  now = ktime_get_ns()
  b = src_buckets[ip.src]   # LRU hash
  if b missing: create(tokens=BURST)
  refill(b, now, SRC_RATE, SRC_BURST)
  if b.tokens < 1:
      stats.src_drop++ ; DROP
  g = global_bucket[this_cpu]
  refill(g, now, GLOBAL_RATE/ncpu)
  if g.tokens < 1:
      stats.glob_drop++ ; DROP
  b.tokens--; g.tokens--
  stats.pass++ ; PASS
```

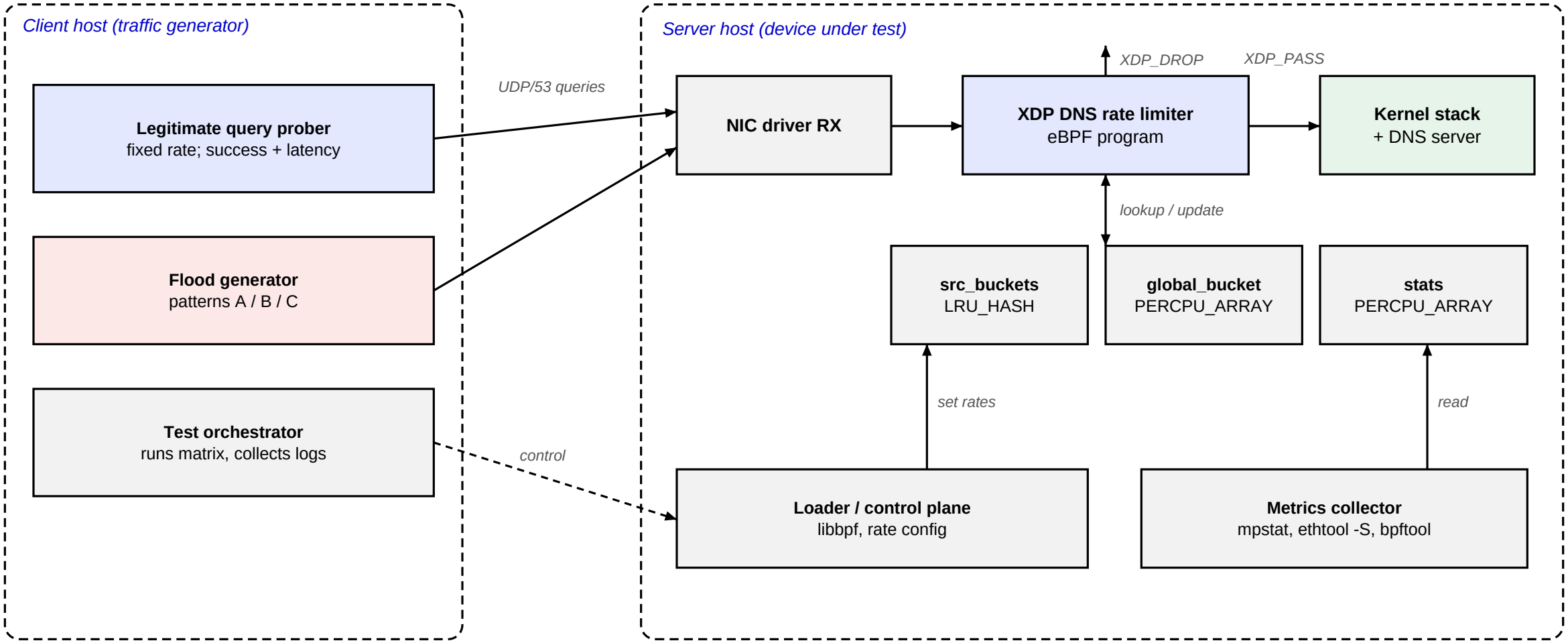
Optimization Proposed

Before (naive / baseline) versus after (proposed XDP design).

Aspect	Before: nftables / naive	After: proposed XDP design
Drop point	After sk_buff allocation and stack traversal	At the driver RX hook, before skb allocation (XDP_DROP)
Parsing	Full DNS name/record parsing	Header-only, verifier-bounded checks (port 53, length, QR)
Per-source state	Unbounded table that can be filled by many sources	Fixed-size LRU hash map; old entries evicted automatically
Global limit	Single shared counter, cache-line contention	Per-CPU token buckets, each holding global rate / CPU count
Burst handling	Hard cap drops legitimate bursts	Token bucket with a configurable burst allowance
Observability	Log lines or coarse rule counters	Per-CPU counters: pass, src_drop, glob_drop, parse_drop
Portability	Needs only netfilter	Native mode on XDP-capable NIC; generic mode documented as fallback

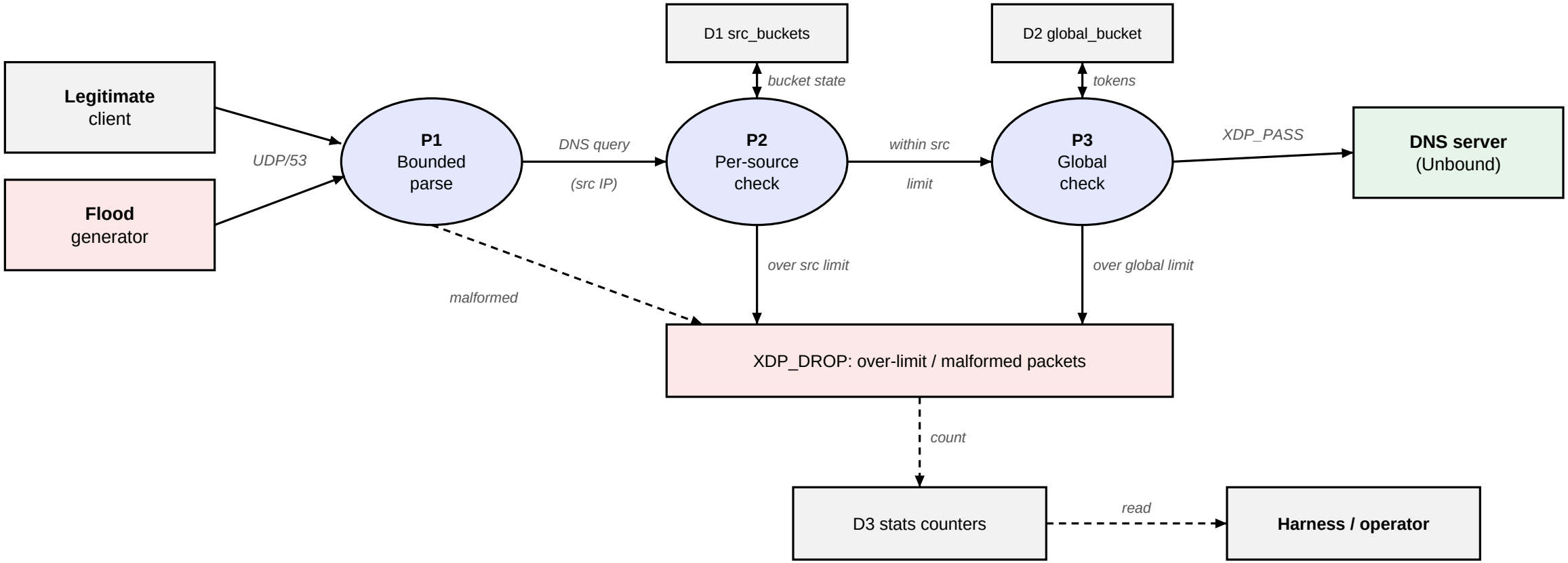
Solution Architecture & Design

Component view of the lab host pair and the XDP rate-limiting path.



Data Flow Diagram

Level-1 flow of a DNS packet through the XDP filter; dashed items are counters and control.



Non-DNS or non-UDP traffic is passed through untouched (XDP_PASS).

Deployment Timeline

Iterative, phase-by-phase delivery; every phase is committed to the private GitHub repo for mentor review.

1. **Lab setup & baselines**
   Host pair, kernel/NIC check (native vs generic), DNS server, no-filter and nftables configs
2. **XDP prototype**
   Bounded parser plus per-source token bucket; tested with small crafted packet sets
3. **Global limit & stats**
   Per-CPU global buckets, counters, loader with rate configuration
4. **Harness & patterns**
   Legitimate prober, flood generator for patterns A/B/C, metric capture scripts
5. **Benchmark & analysis**
   3 configs x 3 patterns x repeated runs; throughput, CPU, drops, legit success
6. **Report & demo**
   PDF in the repo, results plots, prototype demo with pseudocode to the mentor

Bibliography

[1] T. Hoiland-Jorgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, D. Miller. "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel." In: Proc. ACM CoNEXT 2018, pp. 54-66.

[2] G. Bertin. "XDP in practice: integrating XDP into our DDoS mitigation pipeline." NetDev 2.1, 2017 (Cloudflare).

[3] P. Mockapetris. "Domain Names - Implementation and Specification." RFC 1035, IETF, 1987.

[4] J. Damas, F. Neves. "Preventing Use of Recursive Nameservers in Reflector Attacks." RFC 5358 (BCP 140), IETF, 2008.

[5] The Linux Kernel documentation. "BPF Documentation." https://docs.kernel.org/bpf/

Thank You for Your Attention!